

Documentación del módulo persistencia de eShop Core

Instituto Superior de Formación Técnica N. º 179 Dr. Carlos Pellegrini

Carrera: Tecnicatura Superior en Análisis de Sistemas

Materia: Algoritmos y Estructura de Datos III

Docente: Hansen Andrés

Ayudante de cátedra: Ríos Emmanuel

Tema: Desarrollo de aplicación con TAD

Estudiantes: Trujillo Sebastián y Sanoguera Agustin

Funcionamiento e integración del módulo de persistencia

El módulo BD/persistencia.py conserva la información del sistema en BD/tablas.db mediante SQLite. Permite recuperar clientes, productos, reservas, compras y avisos después de cerrar el servidor. Es el punto de acceso a los datos para los módulos del backend: recibe parámetros de Python, ejecuta SQL y devuelve diccionarios, listas, identificadores o valores de control.

La persistencia no maneja pantallas ni solicitudes HTTP y no manipula los nodos de listas, colas, pilas o árboles. Esas responsabilidades pertenecen a los demás módulos. Su papel consiste en guardar los cambios duraderos y proporcionar los registros con los que se reconstruyen las estructuras en memoria. Este documento describe el código vigente y sus flujos, sin ejecutar operaciones sobre la base de datos.

Conexión y organización de los datos

Ubicación de los archivos y apertura de conexiones

RUTA_BD se calcula a partir de la ubicación del propio módulo y apunta a tablas.db dentro de BD. RUTA_DDL apunta a BD/ddl.sql. Por eso la selección de la base no depende de la carpeta desde la que se inicia el sitio.

conectar() abre una conexión SQLite, activa PRAGMA foreign_keys = ON y establece sqlite3.Row como fábrica de filas. Con esta configuración los resultados se pueden consultar por nombre de columna y convertir mediante dict(fila). Cada operación que abre su propia conexión la cierra en un bloque finally, aun cuando ocurra un error.

Las consultas utilizan marcadores ? para enviar valores por separado del texto SQL. El administrador de contexto with conexion confirma una transacción abierta cuando finaliza correctamente y la revierte si una excepción sale del bloque. Ese

contexto no cierra la conexión; el cierre explícito se realiza en finally. guardar_aviso es una excepción deliberada al patrón de apertura: recibe una conexión ya abierta para participar en la transacción de quien lo llama.

Tablas utilizadas y vínculos

clientes almacena id_cliente, nombre, apellido y dni. El identificador interno se utiliza en las relaciones; el DNI permite recuperar un cliente desde el portal. productos contiene id_producto, nombre, cantidad_disponible y precio.

esperas relaciona un cliente con un producto agotado e incluye id_espera y fecha_espera. transacciones guarda la cabecera de una compra, con cliente, fecha y total. detalle_transaccion vincula cada compra con sus productos y conserva cantidad y precio_unitario.

notificaciones guarda los avisos de reposición asociados a un cliente y un producto, con fecha y mensaje. Sus registros permanecen disponibles después de que la espera atendida sea eliminada.

El DDL establece claves primarias, claves foráneas y restricciones CHECK, como stock y precios no negativos y cantidades de detalle mayores que cero. No define borrado en cascada. Por ello eliminar un producto relacionado con esperas, detalles o notificaciones puede ser rechazado por SQLite. Las validaciones de la aplicación complementan esas restricciones.

Inicialización y adaptación de clientes

inicializar_bd() llama primero a migrar_clientes_dni(), lee ddl.sql en UTF-8 y ejecuta el script con executescript. Las sentencias CREATE TABLE IF NOT EXISTS permiten crear tablas faltantes sin borrar las existentes. Esto no equivale a actualizar

automáticamente cualquier esquema antiguo: la adaptación del campo email se resuelve mediante la migración específica.

migrar_clientes_dni() inspecciona PRAGMA table_info(clientes). Si la tabla no existe o ya no contiene email, termina sin transformar datos ni crear un respaldo. Si detecta el esquema anterior, utiliza la API backup de SQLite para conservar una copia con nombre respaldo_antes_dni y una marca temporal.

La migración desactiva temporalmente la aplicación de claves foráneas en su conexión y, dentro de BEGIN IMMEDIATE, crea clientes_nuevos, copia los identificadores y nombres anteriores, elimina la tabla antigua y renombra la nueva. Comprueba las relaciones con foreign_key_check antes de confirmar. Los clientes heredados quedan con dni NULL, sin asignarles documentos ficticios.

Esta copia de la base pertenece únicamente al proceso de migración. No es la misma operación que devolver copias de diccionarios ni se genera en cada alta, consulta o compra.

Persistencia de productos

leer_productos() devuelve todos los productos como una lista de diccionarios ordenada por id_producto. inventario.cargar_inventario() valida estos registros y los incorpora a una lista enlazada. consultar_producto(id_producto) devuelve un diccionario o None; también permite sincronizar el inventario después de una compra confirmada.

insertar_producto(nombre, cantidad_disponible, precio) inserta una fila y devuelve cursor.lastrowid. El módulo inventario utiliza ese ID definitivo al incorporar el nuevo producto al TDA Lista. La validación completa del formato del producto se realiza antes, mediante producto.crear_producto.

modificar_producto(id_producto, nombre, cantidad_disponible, precio, id_espera=None) actualiza los datos y devuelve True si encuentra el producto, o False si no existe. Usa BEGIN IMMEDIATE para mantener las comprobaciones y la escritura en la misma transacción.

Si recibe id_espera, comprueba que el stock guardado sea cero, que la nueva cantidad sea positiva y que ese identificador corresponda a la primera espera del producto en SQLite. Luego actualiza el producto, guarda el aviso y elimina la espera en una sola transacción. Sin id_espera, no decide por sí mismo atender una cola: la decisión y la selección del cliente corresponden al inventario y al módulo de esperas.

eliminar_producto(id_producto) ejecuta DELETE y devuelve si eliminó alguna fila. Si una clave foránea impide el borrado, propaga el error. inventario.py retira el producto de su lista únicamente después de confirmar el resultado en SQLite.

Ingreso y recuperación de clientes

leer_clientes() devuelve los campos id_cliente, nombre, apellido y dni, ordenados por ID. consultar_cliente(id_cliente) busca un registro concreto y devuelve un diccionario o None. Estas consultas permiten verificar clientes y obtener los datos requeridos por otras operaciones.

normalizar_dni(dni) exige texto, elimina puntos y espacios y acepta únicamente siete u ocho dígitos ASCII. El DNI se conserva como texto. normalizar_nombre(valor, campo) elimina espacios sobrantes, exige entre dos y cincuenta caracteres y admite letras y espacios. Aunque su descripción interna menciona guiones y apóstrofes, la condición ejecutada actualmente los rechaza; el comportamiento documentado corresponde a esa condición.

ingresar_cliente(dni, nombre=None, apellido=None) normaliza el documento e inicia BEGIN IMMEDIATE. Si el DNI existe, devuelve el registro guardado sin sustituir su nombre o apellido por nuevos valores. Si no existe y no se proporcionan ambos datos personales, distingue dos situaciones: con los dos parámetros omitidos devuelve None; si se intenta registrar con uno faltante, la validación correspondiente produce un error.

Cuando recibe los datos de un nuevo cliente, valida nombre y apellido, inserta la fila y devuelve el registro creado. La restricción UNIQUE del DNI impide guardar dos clientes con el mismo documento. El servidor utiliza None para solicitar los datos de alta y, cuando obtiene un cliente, administra su sesión por separado. La persistencia no guarda el token de ingreso.

Reservas y clientes en espera

existe_espera(id_cliente, id_producto) informa si existe una espera activa para ese par. No inserta ni elimina datos. insertar_espera(id_cliente, id_producto, fecha_espera) realiza sus propias comprobaciones dentro de BEGIN IMMEDIATE: exige cliente y producto existentes, stock exactamente cero y ausencia de otra espera activa de ese mismo cliente para el producto.

Si la operación es válida, insertar_espera guarda los datos y devuelve el nuevo id_espera. esperas.registrar_espera() utiliza ese resultado para agregar el registro al final del TDA Cola. La fecha llega desde esperas.py; la persistencia de reservas no la genera nuevamente.

leer_esperas(id_producto=None) devuelve una lista de diccionarios ordenada por id_espera. Sin filtro recupera todas las reservas para reconstruir las colas al iniciar;

con filtro obtiene únicamente las de un producto. Ese orden conserva la prioridad de llegada dentro de cada cola.

eliminar_espera(id_espera) permite borrar un registro de forma independiente y devuelve si existía. No actualiza por sí sola el TDA Cola. El flujo normal de reposición usa la eliminación integrada en modificar_producto, porque también debe confirmar el stock y el aviso. La ausencia de duplicados se controla en insertar_espera dentro de la transacción; el DDL actual no declara UNIQUE sobre el par cliente y producto.

Avisos de reposición

guardar_aviso(conexion, id_espera, nombre_producto) busca la espera indicada en la conexión recibida. Si ya no existe, lanza ValueError. Si existe, inserta una notificación con los identificadores del cliente y producto, una fecha UTC en formato ISO 8601 y un mensaje de disponibilidad. Devuelve el texto del aviso y no confirma ni cierra la conexión.

El uso de la misma conexión permite que stock, aviso y eliminación de espera se confirmen juntos. leer_notificaciones(id_cliente) devuelve hasta veinte avisos del cliente, del más reciente al más antiguo por id_notificacion. El servidor expone esa información al portal, que la muestra en el carrito. No se trata de un envío de correo ni de una reserva automática de unidades.

Compras y detalle de transacciones

Confirmación de una compra

guardar_compra(compra) recibe la compra preparada por transacciones.py. Dentro de BEGIN IMMEDIATE vuelve a comprobar que el cliente exista, que los productos no estén repetidos, que cada producto siga disponible en la cantidad solicitada

y que el precio actual coincida con precio_unitario. Así detecta cambios ocurridos después de preparar el carrito.

El total se recalcula a partir de los precios de SQLite y las cantidades, utilizando Decimal antes de convertir el resultado al tipo numérico utilizado para guardarlo. Se rechazan resultados no finitos. No se utiliza directamente el total enviado en el diccionario como valor definitivo.

Después se inserta la cabecera en transacciones, se obtiene su ID, se guarda cada detalle y se descuenta la cantidad comprada del stock. Todo queda dentro de la misma transacción. Si una validación o escritura falla, no se confirma parcialmente una compra con solo algunos productos. La función devuelve id_transaccion al terminar correctamente.

La validación de la estructura de entrada, las cantidades enteras positivas, los límites y el agrupamiento de productos corresponde primero a transacciones.preparar_compra(). guardar_compra no sustituye ese contrato de entrada. Después del guardado, transacciones.py sincroniza el inventario, apila la compra y actualiza el árbol de popularidad.

Lectura del historial persistente

leer_transacciones() inicia una transacción de lectura con BEGIN, recupera las cabeceras por id_transaccion ascendente y agrega a cada una una lista productos. Para formar los detalles une detalle_transaccion con productos y los ordena por id_detalle.

El resultado incluye id_producto, nombre, cantidad y precio_unitario en cada detalle. El precio unitario es el conservado en la compra; el nombre se toma de la tabla productos en el momento de la consulta. Por tanto, un cambio posterior del nombre

puede aparecer al reconstruir el historial. transacciones.cargar_pila() apila los registros en orden ascendente y deja la compra más reciente en la cima.

Reversión interna conservada en el código

revertir_ultima_compra(id_transaccion) comprueba que el ID recibido sea el máximo del historial. Si coincide, restituye el stock de los detalles, controla que no exceda el máximo entero de SQLite y elimina los detalles y la cabecera dentro de una misma transacción.

Si la restitución hace que un producto con stock cero vuelva a tener unidades y existe una espera, guarda un aviso y elimina la primera espera de ese producto. Devuelve los registros atendidos para que transacciones.deshacer_transaccion() sincronice lista, colas, pila y popularidad.

Esta función permanece como capacidad interna del código. La API actual no ofrece una operación para deshacer compras y el portal no presenta esa opción. Documentar su existencia no implica habilitarla como parte del flujo de compra del cliente.

Popularidad y relación con el árbol BST

leer_popularidad() consulta detalle_transaccion, agrupa por id_producto y calcula SUM(cantidad) AS popularidad. Devuelve los resultados ordenados por producto. La métrica es la cantidad de unidades vendidas, no la cantidad de compras ni el stock disponible.

Por ejemplo, si un producto aparece con cantidades dos y tres en compras distintas, su popularidad es cinco. Un producto sin detalles de venta no aparece en esa consulta. recomendaciones.cargar_popularidad() usa los resultados para construir el árbol BST con claves (popularidad, id_producto) y el diccionario de conteos.

El árbol y sus nodos no se almacenan en tablas. Se reconstruyen a partir del historial de ventas y se actualizan en memoria al confirmar compras. Aunque un comentario del DDL mencione COUNT(*), la consulta ejecutada en persistencia.py utiliza SUM(cantidad); esa es la regla vigente.

Relación con los demás módulos

Servidor y portal de compras

servidor.py inicia la base antes de cargar los estados del sistema. También llama directamente a las funciones de clientes y notificaciones. Convierte las solicitudes HTTP en operaciones del backend y devuelve respuestas JSON o mensajes de error. El navegador no abre tablas.db ni envía SQL.

El recorrido habitual es portal del cliente, API del servidor, módulo encargado de la operación y persistencia. Las respuestas regresan por el camino inverso. Las sesiones se conservan en memoria del servidor y el carrito es un estado de la interfaz; las filas de clientes, esperas, notificaciones y compras sí permanecen en SQLite.

Inventario, productos y TDA Lista

inventario.py utiliza las lecturas y escrituras de productos para coordinar SQLite con su lista enlazada. producto.py establece las reglas de los diccionarios de producto. La persistencia devuelve datos comunes y no importa inventario.py, producto.py ni las primitivas de listas. Esta dirección de dependencias permite acceder a la base sin depender de una pantalla o de una estructura enlazada concreta.

Esperas y TDA Cola

esperas.py utiliza leer_esperas, insertar_espera y consultar_cliente. Reconstruye las colas, incorpora reservas después de guardarlas y prepara la atención del primero. inventario.py coordina la reposición con modificar_producto. Una vez confirmado el

cambio duradero, se retira al cliente del frente de su cola en memoria. persistencia.py no llama a arribo ni a atencion.

Transacciones, TDA Pila y recomendaciones

transacciones.py prepara y valida las compras, llama a guardar_compra y reconstruye su pila mediante leer_transacciones. recomendaciones.py recupera los totales de unidades con leer_popularidad y organiza el BST. La persistencia no apila, desapila ni inserta nodos de árbol: conserva las filas necesarias para recuperar esos estados.

Las estructuras en memoria son coordinadas por los módulos de negocio. El Lock compartido del servidor evita que sus solicitudes modifiquen esos estados simultáneamente dentro del mismo proceso. Las transacciones SQLite protegen los cambios duraderos. Son mecanismos complementarios y no convierten la memoria y el archivo de base en una única transacción.

Flujos de funcionamiento del sistema

Inicio y recuperación

Al iniciar, el servidor llama a inicializar_bd(). Después inventario.cargar_inventario() obtiene los productos; esperas.cargar_colas() recupera las reservas; recomendaciones.cargar_popularidad() calcula las unidades vendidas; y transacciones.cargar_pila() recupera las compras. Cada módulo organiza su propio TDA a partir de los datos recibidos.

Al reiniciar se reconstruyen esos estados desde SQLite. Las sesiones se crean nuevamente y el cliente debe identificarse por DNI. La persistencia de datos y la permanencia de una sesión son cuestiones diferentes.

Registro e ingreso del cliente

El portal solicita el DNI y el servidor llama a ingresar_cliente. Si el documento existe, recupera al cliente. Si no existe y faltan los datos de alta, solicita nombre y apellido. Al completar el registro se confirma la nueva fila y el servidor crea la sesión que identificará las siguientes reservas y compras.

Reserva y reposición de un producto

El cliente selecciona Reservar en un producto agotado. El servidor obtiene su ID desde la sesión y esperas.py solicita insertar_espera. SQLite valida y guarda; luego la reserva ingresa al final de la cola. El portal consulta sus reservas para mostrar su posición.

Al modificar el stock de cero a positivo, inventario.py consulta al primero de la cola y envía su id_espera a modificar_producto. SQLite verifica el orden y confirma juntos la reposición, el aviso y la eliminación de la espera. Luego inventario actualiza su lista y esperas retira al primero de la cola. El cliente consulta el aviso a través del servidor.

El flujo actual atiende a un cliente por esa transición de stock, aunque se repongan varias unidades. El aviso no descuenta existencias ni confirma una venta. La compra se realiza en una operación posterior.

Compra y actualización de las estructuras

El cliente confirma los productos del carrito. transacciones.py prepara cantidades y precios; guardar_compra vuelve a comprobar los datos vigentes y confirma cabecera, detalles y descuento de stock. Después se actualiza la lista del inventario, se incorpora la compra a la pila y se incrementa la popularidad de cada producto según las unidades vendidas.

Si SQLite rechaza la compra, esas actualizaciones en memoria no deben ejecutarse. Si el proceso se interrumpe después de confirmar SQLite, el archivo ya contiene los datos duraderos y el siguiente inicio permite reconstruir las estructuras. No existe en el módulo una transacción distribuida que revierta automáticamente un error posterior de memoria.

Tratamiento de errores y alcance

Las validaciones de negocio generan ValueError y las infracciones de restricciones pueden producir excepciones sqlite3. La persistencia las propaga; el servidor y los módulos llamadores determinan cómo informar el problema. Los bloques finally cierran las conexiones abiertas por cada función y las transacciones evitan confirmar solo una parte de las operaciones agrupadas.

Convertir filas en diccionarios produce objetos de Python independientes de la fila consultada. Esa conversión facilita el intercambio con los demás módulos y no duplica registros en la base. También debe distinguirse de un respaldo del archivo y de los estados temporales de los TDA.

El módulo proporciona almacenamiento y recuperación. La presentación pertenece al frontend, el manejo HTTP al servidor, la validación del flujo a los módulos de negocio y las operaciones de estructuras a sus TDA. Esa distribución explica cómo listas, colas, pila y BST se relacionan con una misma base sin que persistencia.py deba manipular sus nodos.